Codilar_CategoryCache

Magento 2.4.9 module: cached category details (ID, name, URL). How to see it working, and a line-by-line explanation of every file.

1. What the module does

Requirement	How it is met
Fetch category ID, name and URL	`CategoryDetailsInterface::get($categoryId, $storeId)` returns `['id','name','url']`
Cache the result for one hour	Saved with lifetime `CACHE_LIFETIME = 3600` seconds
Category ID in the cache key	Key: `CODILAR_CATEGORY_DETAILS_<categoryId>_<storeId>`
Clear the cache for one category	`clean($categoryId)`, through code or `bin/magento codilar:category:cache:clean <id>`. Also runs automatically when a category is saved or deleted.
Show that cached data is reused	`bin/magento codilar:category:get <id>` prints CACHE MISS / CACHE HIT with timings. Also logged to `var/log/category_cache.log`.

Request flow

1. Caller
get(3, 1)

2. Build key
CODILAR_CATEGORY_DETAILS_3_1

3a. In cache?
Yes: return the stored array (HIT, <1 ms, no database query)

3b. Not in cache
Load from the category repository, then save for 3600 s with tag CODILAR_CATEGORY_DETAILS_3 (MISS)

4. Clean
clean(3) removes every entry tagged CODILAR_CATEGORY_DETAILS_3

File map

File	Role
`registration.php`, `etc/module.xml`	Module registration and load order
`etc/cache.xml`, `Model/Cache/Type/CategoryDetails.php`	Separate cache type, managed in Cache Management
`Api/CategoryDetailsInterface.php`	Public service contract and the 1-hour lifetime
`Model/CategoryDetails.php`	Cache read/write, database fallback, per-category clean
`Observer/CleanCategoryDetailsCache.php`, `etc/events.xml`	Clears the cache automatically on category save/delete
`etc/di.xml`	Interface preference, dedicated logger, CLI registration
`Console/Command/GetCommand.php`, `CleanCommand.php`	CLI demo commands

2. How to see it working

Run every command from the Magento root: `/var/www/html/cron-cache-queue`.

Step 1. Install and enable the module

```
bin/magento module:enable Codilar_CategoryCache
bin/magento setup:upgrade
bin/magento cache:enable codilar_category_details
bin/magento cache:status | grep codilar      # expect: codilar_category_details: 1
```

In production mode, also run `bin/magento setup:di:compile` to generate the Proxy classes.

Step 2. Find a category ID

Admin > Catalog > Categories: the ID appears next to the category name in the tree, e.g. *Test Category (ID: 3)*.

Step 3. First fetch: MISS, then HITs

```
$ bin/magento codilar:category:get 3 --times=3
Call #1: CACHE MISS - loaded from database (92.03 ms)
  ID: 3 | Name: Test Category | URL: http://localhost:8083/test-category.html
Call #2: CACHE HIT (0.27 ms)
  ID: 3 | Name: Test Category | URL: http://localhost:8083/test-category.html
Call #3: CACHE HIT (0.15 ms)
  ID: 3 | Name: Test Category | URL: http://localhost:8083/test-category.html
```

Call #1 queried the database and saved the result. Calls #2 and #3 returned the same data from the cache in a fraction of a millisecond, without touching the database.

Step 4. The cache is shared between requests

```
$ bin/magento codilar:category:get 3 --times=1
Call #1: CACHE HIT (1.66 ms)
```

This is a new PHP process, so nothing is left in memory from Step 3. The data is coming from Magento's cache storage. A storefront page, a REST call or a cron job using the service would get the same hit.

Step 5. Clear one category, then fetch again

```
$ bin/magento codilar:category:cache:clean 3
Cleared cached details for category 3.

$ bin/magento codilar:category:get 3
Call #1: CACHE MISS - loaded from database (45.94 ms)
Call #2: CACHE HIT (0.31 ms)
```

After the clean, the next call misses and saves the entry again. Cached entries for other categories are not touched.

Step 6. Watch the log

```
$ tail -f var/log/category_cache.log
categoryCache.DEBUG: Category details cache MISS: CODILAR_CATEGORY_DETAILS_3_1
categoryCache.DEBUG: Category details cache HIT: CODILAR_CATEGORY_DETAILS_3_1
categoryCache.DEBUG: Category details cache cleaned for category 3
```

The log shows each key, including the category ID. Debug logging only runs in developer mode (`bin/magento deploy:mode:show`).

Step 7. Automatic clean on category save

1. Run `bin/magento codilar:category:get 3` so the entry is cached (HIT on call #2).
2. In the admin, rename category 3 and click **Save**.
3. Run `bin/magento codilar:category:get 3` again. Call #1 should be a **MISS** showing the **new name**, and the log should contain a "cache cleaned for category 3" line.

> This step has not been run yet. Steps 3–6 were run on this installation and produced the output shown above.

Step 8. Toggle or flush from the admin

Admin > System > Cache Management lists **Codilar Category Details**. If you disable it, every call becomes a MISS because the cache is skipped. *Flush* clears all categories at once (equivalent to `bin/magento cache:clean codilar_category_details`).

Step 9. Use it from your own code

```
public function __construct(
    private readonly \Codilar\CategoryCache\Api\CategoryDetailsInterface $categoryDetails
) {}

$details = $this->categoryDetails->get(3);   // ['id' => 3, 'name' => '...', 'url' => '...']
$this->categoryDetails->clean(3);            // clear category 3
```

In storefront code (blocks, controllers, view models), leave out the store ID and the current store is used.

3. Line-by-line explanation

Files are ordered from the foundation (registration, config) to the logic (service) to the tooling (commands). Line numbers match the files on disk. Adjacent lines that do one thing together are explained as a group.

1 `registration.php`

> Tells Magento that this folder is a module and what it is called. Magento's Composer autoloader runs every registration.php it finds under app/code at bootstrap.

Lines	Code	What it does
1	`1 <?php`	Opens a PHP block. PHP-only files leave out the closing `?>` so that trailing whitespace can't be sent as output by accident.

2–4	`2 /**`
	`3  * Registers the Codilar_CategoryCache module.`
	`4  */`
		Docblock describing the file.

5	`5 declare(strict_types=1);`	Strict typing: PHP throws a TypeError instead of silently converting values (for example the string "3" to an int). Every PHP file in the module has this line.
6	`6`	Blank line for readability.
7	`7 use Magento\Framework\Component\ComponentRegistrar;`	Imports the `ComponentRegistrar` class so it can be used by its short name.
8	`8`	Blank line.
9	`9 ComponentRegistrar::register(ComponentRegistrar::MODULE, 'Codilar_CategoryCache', __DIR__);`	Registers a component of type MODULE named `Codilar_CategoryCache` (Vendor_Module), located in this directory (`__DIR__`). This name is the one `bin/magento module:enable` and app/etc/config.php refer to.

2 `etc/module.xml`

> Declares the module to Magento and sets the order in which modules load.

Lines	Code	What it does
1	`1 <?xml version="1.0"?>`	XML declaration.
2–3	`2 <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"` `3 xsi:noNamespaceSchemaLocation="urn:magento:framework:Module/etc/module.xsd">`	Root `<config>` node. `noNamespaceSchemaLocation` points to module.xsd, which Magento uses to validate the file (IDEs use it for autocompletion).
4	`4     <module name="Codilar_CategoryCache">`	Module name. It must match the name in registration.php exactly.

5–8	`5         <sequence>`
	`6             <module name="Magento_Store"/>`
	`7             <module name="Magento_Catalog"/>`
	`8         </sequence>`
		`<sequence>` : Magento_Store and Magento_Catalog load before this module, so their config, DI and events are already in place when ours merge on top. We depend on them for the store manager, store emulation and the category repository.

9–10	`9     </module>` `10 </config>`	Closing tags.

3 `etc/cache.xml`

> Creates a separate **cache type**. It appears in Admin > System > Cache Management and in `bin/magento cache:status` , so

it can be enabled, disabled or flushed on its own without touching the rest of the cache.

Lines	Code	What it does
1	`<?xml version="1.0"?>`	XML declaration.
2	`<!-- Shows up in System > Cache Management and in bin/magento cache:status -->`	Comment noting where this cache type shows up.
3–4	`<config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="urn:magento:framework:Cache/etc/cache.xsd">`	Root node, validated against cache.xsd.
5–6	`<type name="codilar_category_details" translate="label,description" instance="Codilar\CategoryCache\Model\Cache\Type\CategoryDetails">`	`name` is the cache type ID used by the CLI and cache state (`codilar_category_details`). `instance` is the PHP class that reads and writes this type. `translate` marks the label and description as translatable.
7–8	`<label>Codilar Category Details</label>` `<description>Category ID, name and URL, cached per category and store view.</description>`	Label and description shown in the Cache Management grid.
9–10	`</type>` `</config>`	Closing tags.

4 Model/Cache/Type/CategoryDetails.php

The PHP class behind the cache type. It wraps Magento's cache storage and adds the tag `CODILAR_CATEGORY_DETAILS` to every entry it saves. Flushing the type from the admin cleans that tag, so only our entries are removed.

Lines	Code	What it does
1–6	`<?php` `/**` ` * Dedicated cache type so the entries can be toggled/flushed on their own.` ` */` `declare(strict_types=1);`	PHP open tag, docblock, strict types, blank line.
7	`namespace Codilar\CategoryCache\Model\Cache\Type;`	Namespace. It mirrors the folder path (PSR-4: Codilar/CategoryCache/Model/Cache/Type).
8–11	`use Magento\Framework\App\Cache\Type\FrontendPool;` `use Magento\Framework\Cache\Frontend\Decorator\TagScope;`	Imports: `FrontendPool` provides the configured cache storage (file, Redis, etc.). `TagScope` is a decorator that adds a fixed tag to every save and limits clean() to that tag.
12–13	`class CategoryDetails extends TagScope` `{`	The class extends TagScope, so it gets `load()`, `save()`, `remove()` and `clean()` without any extra code.
14	`public const TYPE_IDENTIFIER = 'codilar_category_details';`	Constant holding the cache type ID. It must match `name` in cache.xml. The service uses it to check whether the type is enabled.
15–17	`/** Applied to every entry of this type; used by "flush this cache type". */` `public const CACHE_TAG = 'CODILAR_CATEGORY_DETAILS';`	The tag added to every entry of this type. It is what "Flush" on this type in Cache Management cleans.
18–21	`/**` `* @param FrontendPool $cacheFrontendPool` `*/`	Constructor docblock.
22–23	`public function __construct(FrontendPool $cacheFrontendPool)` `{`	Constructor. Magento's dependency injection (DI) passes in the FrontendPool automatically.

Lines	Code	What it does
24	`24 parent::__construct($cacheFrontendPool->get(self::TYPE_IDENTIFIER), self::CACHE_TAG);`	`$cacheFrontendPool->get(TYPE_IDENTIFIER)` returns the storage configured for this type (the default storage unless env.php overrides it). The parent TagScope wraps it with our type tag.
25–26	`25 }` `26 }`	Closing braces.

5 Api/CategoryDetailsInterface.php

> The public contract (service interface). Other code depends on this interface, not on the concrete class, so the implementation can be replaced through a DI preference without changing the callers.

Lines	Code	What it does

1–6	`1 <?php`
	`2 /**`
	`3 * Cached lookup of basic category details.`
	`4 */`
	`5 declare(strict_types=1);`
	`6`
		PHP open tag, docblock, strict types, blank line.

7	`7 namespace Codilar\CategoryCache\Api;`	Namespace. Interfaces that other modules may use go in the `Api` folder by convention.

8–10	`8`
	`9 use Magento\Framework\Exception\NoSuchEntityException;`
	`10`
		Imports NoSuchEntityException, which is referenced in the @throws tag.

11–12	`11 interface CategoryDetailsInterface` `12 {`	Interface declaration.
13–14	`13 /** Cache lifetime in seconds (1 hour). */` `14 public const CACHE_LIFETIME = 3600;`	**Requirement: cache for one hour.** `CACHE_LIFETIME = 3600` seconds. Putting it on the interface makes the lifetime part of the contract and keeps it in one place.

15–23	`15`
	`16 /**`
	`17 * Get category details, served from cache when available.`
	`18 *`
	`19 * @param int $categoryId`
	`20 * @param int|null $storeId Defaults to the current store`
	`21 * @return array{id: int, name: string, url: string}`
	`22 * @throws NoSuchEntityException`
	`23 */`
		Docblock for get(). The return shape `array{id, name, url}` is documented for IDEs and static analysis. A missing category throws NoSuchEntityException.

24	`24 public function get(int $categoryId, ?int $storeId = null): array;`	**Requirement: fetch category ID, name and URL.** `get()` takes the category ID and an optional store view ID. If no store is given, the current store is used.

25–30	`25`
	`26 /**`
	`27 * Whether the last get() call was answered from cache.`
	`28 *`
	`29 * @return bool`
	`30 */`
		Docblock for wasLastHit().

31	`31 public function wasLastHit(): bool;`	Reports whether the previous get() call came from the cache. It exists so the demo command can show HIT or MISS.

32–38	`32`
	`33 /**`
	`34 * Remove cached details of one category (all store views).`
	`35 *`
	`36 * @param int $categoryId`
	`37 * @return void`
	`38 */`
		Docblock for clean().

Lines	Code	What it does
39	`39 public function clean(int $categoryId): void;`	**Requirement: clear the cache for one category.** Removes that category's cached entries in every store view.
40	`40 }`	End of interface.

6 `Model/CategoryDetails.php`

> The main service. It looks in the cache first. On a miss it loads the category from the database, builds the result and saves it to the cache for one hour.

Lines	Code	What it does

1–7	`1 <?php`
	`2 /**`
	`3  * Loads category ID, name and URL and caches them for one hour.`
	`4  *`
	`5  * Cache key: CODILAR_CATEGORY_DETAILS_<categoryId>_<storeId>`
	`6  * Cache tag: CODILAR_CATEGORY_DETAILS_<categoryId> (lets clean() drop every store view at once)`
	`7  */`
		PHP open tag and a docblock that records the key and tag formats.

8–11	`8 declare(strict_types=1);`
	`9`
	`10 namespace Codilar\CategoryCache\Model;`
	`11`
		Strict types and namespace.

12–18	`12 use Codilar\CategoryCache\Api\CategoryDetailsInterface;`
	`13 use Codilar\CategoryCache\Model\Cache\Type\CategoryDetails as CacheType;`
	`14 use Magento\Catalog\Api\CategoryRepositoryInterface;`
	`15 use Magento\Framework\App\Cache\StateInterface;`
	`16 use Magento\Framework\Serialize\SerializerInterface;`
	`17 use Magento\Store\Model\StoreManagerInterface;`
	`18 use Psr\Log\LoggerInterface;`
		Imports. `CacheType` is an alias for our cache type class, to avoid a clash with this class's own name (CategoryDetails). `StateInterface` tells us whether a cache type is enabled. `CategoryRepositoryInterface` is Magento's service for loading categories. `SerializerInterface` turns arrays into strings for storage (JSON by default). `LoggerInterface` is the PSR-3 logger.

19–21	`19`
	`20 class CategoryDetails implements CategoryDetailsInterface`
	`21 {`
		The class implements our interface. di.xml maps the interface to this class.

22	`22     private const KEY_PREFIX = 'CODILAR_CATEGORY_DETAILS_';`	`KEY_PREFIX` is shared by the cache key and the per-category tag. A unique prefix keeps us from colliding with other modules' cache entries.

23–27	`23`
	`24     /**`
	`25      * @var bool`
	`26      */`
	`27     private bool $lastHit = false;`
		`$lastHit` holds the HIT/MISS result of the latest get() call for wasLastHit().

28–36	`28`
	`29     /**`
	`30      * @param CacheType $cache`
	`31      * @param StateInterface $cacheState`
	`32      * @param CategoryRepositoryInterface $categoryRepository`
	`33      * @param StoreManagerInterface $storeManager`
	`34      * @param SerializerInterface $serializer`
	`35      * @param LoggerInterface $logger`
	`36      */`
		Constructor docblock.

37–45	`37     public function __construct(`
	`38         private readonly CacheType $cache,`
	`39         private readonly StateInterface $cacheState,`
	`40         private readonly CategoryRepositoryInterface $categoryRepository,`
	`41         private readonly StoreManagerInterface $storeManager,`
	`42         private readonly SerializerInterface $serializer,`
	`43         private readonly LoggerInterface $logger`
	`44     ) {`
	`45     }`
		Constructor property promotion (PHP 8): each `private readonly` parameter becomes a property. DI supplies all six. The logger is replaced in di.xml with our own file logger.

Lines	Code	What it does
46–50	46 47 /** 48 * @inheritDoc 49 */ 50 public function get(int $categoryId, ?int $storeId = null): array	get() starts here.
51	51 {	Opening brace.
52	52 $storeId ??= (int)$this->storeManager->getStore()->getId();	If the caller did not pass a store ID, the current store's ID is used (`??=` only assigns when the value is null).
53	53 $cacheKey = $this->getCacheKey($categoryId, $storeId);	**Requirement: category ID in the cache key.** Builds `CODILAR_CATEGORY_DETAILS_<categoryId>_<st` e.g. `CODILAR_CATEGORY_DETAILS_3_1` . The store ID is included because the name and URL can differ per store view.
54	54 $cacheEnabled = $this->cacheState->isEnabled(CacheType::TYPE_IDENTIFIER);	Checks whether an admin has disabled this cache type. If so, we skip the cache entirely, as Magento's own cache types do.
55–57	55 56 if ($cacheEnabled) { 57 $cached = $this->cache->load($cacheKey);	`load()` returns the stored string, or `false` if the key does not exist or has expired.
58–63	58 if ($cached !== false) { 59 $this->lastHit = true; 60 $this->logger->debug(sprintf('Category details cache HIT: %s', $cacheKey)); 61 62 return $this->serializer->unserialize($cached); 63 }	**Cache HIT**: record the hit, log it, and return the unserialized array. The database is not queried at all. This is the reuse the demo shows.
64–65	64 } 65	End of the cache-read block.
66–67	66 $this->lastHit = false; 67 $this->logger->debug(sprintf('Category details cache MISS: %s', $cacheKey));	**Cache MISS**: record the miss and log it.
68–69	68 69 $category = $this->categoryRepository->get($categoryId, $storeId);	Loads the category from the database for the given store view, so that store's name and URL are returned. Throws NoSuchEntityException if the ID does not exist (nothing is cached in that case).
70–74	70 $details = [71 'id' => (int)$category->getId(), 72 'name' => (string)$category->getName(), 73 'url' => (string)$category->getUrl(), 74];	Builds the result with just the three requested fields. The casts guarantee the types (int, string, string). `getUrl()` uses the URL rewrite, e.g. `/test-category.html` .
75–76	75 76 if ($cacheEnabled) {	Saves only when the cache type is enabled.
77–82	77 $this->cache->save(78 $this->serializer->serialize($details), 79 $cacheKey, 80 [$this->getCategoryTag($categoryId)], 81 self::CACHE_LIFETIME 82);	`save(data, key, tags, lifetime)` : the array is serialized to a string and stored under the key. It gets the tag `CODILAR_CATEGORY_DETAILS_<id>` (TagScope also adds the type tag automatically). **Lifetime 3600 s**: after one hour the entry expires and the next call is a MISS.
83–86	83 } 84 85 return $details; 86 }	Returns the freshly loaded details.
87–94	87 88 /** 89 * @inheritDoc 90 */ 91 public function wasLastHit(): bool 92 { 93 return $this->lastHit; 94 }	wasLastHit() returns the flag set by the last get() call.

Lines	Code	What it does
95–100	95 96 /** 97 * @inheritDoc 98 */ 99 public function clean(int $categoryId): void 100 {	clean() starts here.
101	101 $this->cache->clean(\Zend_Cache::CLEANING_MODE_MATCHING_TAG, [$this->getCategoryTag($categoryId)]);	**Removes one category's entries.** `CLEANING_MODE_MATCHING_TAG` deletes every entry carrying the tag `CODILAR_CATEGORY_DETAILS_<id>`, i.e. the keys `..._3_1`, `..._3_2` and so on for all store views. Other categories are not affected.
102–104	102 $this->logger->debug(sprintf('Category details cache cleaned for category %d', $categoryId)); 103 } 104	Logs the clean and closes the method.
105–113	105 /** 106 * @param int $categoryId 107 * @param int $storeId 108 * @return string 109 */ 110 private function getCacheKey(int $categoryId, int $storeId): string 111 { 112 return self::KEY_PREFIX . $categoryId . '_' . $storeId; 113 }	getCacheKey(): builds the per-category, per-store key.
114–123	114 115 /** 116 * @param int $categoryId 117 * @return string 118 */ 119 private function getCategoryTag(int $categoryId): string 120 { 121 return self::KEY_PREFIX . $categoryId; 122 } 123 }	getCategoryTag(): builds the per-category tag used for saving and cleaning. End of class.

7 Observer/CleanCategoryDetailsCache.php

> Clears the cache automatically when a category is saved or deleted in the admin. Without it, a renamed category would keep showing its old name until the one-hour entry expired. This goes beyond the original requirements.

Lines	Code	What it does
1–9	1 <?php 2 /** 3 * Drops the cached details when a category is saved or deleted, so a renamed 4 * category or changed URL key does not wait out the 1-hour lifetime. 5 */ 6 declare(strict_types=1); 7 8 namespace Codilar\CategoryCache\Observer; 9	PHP open tag, docblock, strict types, namespace.
10–13	10 use Codilar\CategoryCache\Api\CategoryDetailsInterface; 11 use Magento\Framework\Event\Observer; 12 use Magento\Framework\Event\ObserverInterface; 13	Imports our interface plus Magento's Observer and ObserverInterface.
14–15	14 class CleanCategoryDetailsCache implements ObserverInterface 15 {	Every observer implements ObserverInterface, which defines a single execute() method.

Lines	Code	What it does
16–22	16 /** 17 * @param CategoryDetailsInterface $categoryDetails 18 */ 19 public function __construct(20 private readonly CategoryDetailsInterface $categoryDetails 21) { 22 }	The constructor receives our service (resolved to Model\CategoryDetails through the DI preference).
23–28	23 24 /** 25 * @inheritDoc 26 */ 27 public function execute(Observer $observer): void 28 {	execute() runs when an event this observer listens to is dispatched (see events.xml).
29	29 $category = $observer->getEvent()->getCategory();	Category events pass the category entity as event data under the key `category`.
30–32	30 if ($category && $category->getId()) { 31 $this->categoryDetails->clean((int)$category->getId()); 32 }	If there is a saved category with an ID, its cached details are cleared by calling the same clean() the CLI command uses.
33–34	33 } 34 }	Closing braces.

8 etc/events.xml

> Connects the observer to Magento events. It sits in etc/ (the global area), so it runs for admin saves, REST/API saves, imports and CLI.

Lines	Code	What it does
1–3	1 <?xml version="1.0"?> 2 <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" 3 xsi:noNamespaceSchemaLocation="urn:magento:framework:Event/etc/events.xsd">	XML declaration and root node, validated against events.xsd.
4–6	4 <event name="catalog_category_save_after"> 5 <observer name="codilar_category_cache_clean" instance="Codilar\CategoryCache\Observer\CleanCategoryDetailsCache" /> 6 </event>	On `catalog_category_save_after` (dispatched after a category is saved), run our observer. The observer `name` must be unique for this event.
7–9	7 <event name="catalog_category_delete_after"> 8 <observer name="codilar_category_cache_clean" instance="Codilar\CategoryCache\Observer\CleanCategoryDetailsCache" /> 9 </event>	Same observer on `catalog_category_delete_after`, so deleted categories don't leave stale entries.
10	10 </config>	Closing tag.

9 etc/di.xml

> Dependency injection configuration: which class implements the interface, which logger the service uses, and how the CLI commands are registered.

Lines	Code	What it does
1–3	1 <?xml version="1.0"?> 2 <config xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" 3 xsi:noNamespaceSchemaLocation="urn:magento:framework:ObjectManager/etc/config.xsd">	XML declaration and root node, validated against the ObjectManager config.xsd.
4–5	4 <preference for="Codilar\CategoryCache\Api\CategoryDetailsInterface" 5 type="Codilar\CategoryCache\Model\CategoryDetails"/>	**Preference**: when any class asks for CategoryDetailsInterface, Magento creates Model\CategoryDetails.

Lines	Code	What it does
6–7	6 7 <!-- Dedicated logger: var/log/category_cache.log (debug level shows HIT/MISS) -->	Blank line and a comment.
8–12	8 <virtualType name="Codilar\CategoryCache\Logger\Handler" type="Magento\Framework\Logger\Handler\Debug"> 9 <arguments> 10 <argument name="fileName" xsi:type="string">/var/log/category_cache.log</argument> 11 </arguments> 12 </virtualType>	**virtualType** = a named copy of an existing class with different constructor arguments and no new PHP file. This creates a log handler that writes to `var/log/category_cache.log`. It is based on Magento's Debug handler, so it logs only debug messages, and only in developer mode by default.
13–20	13 <virtualType name="Codilar\CategoryCache\Logger\Logger" type="Magento\Framework\Logger\Monolog"> 14 <arguments> 15 <argument name="name" xsi:type="string">categoryCache</argument> 16 <argument name="handlers" xsi:type="array"> 17 <item name="file" xsi:type="object">Codilar\CategoryCache\Logger\Handler</item> 18 </argument> 19 </arguments> 20 </virtualType>	A second virtualType: a Monolog logger called `categoryCache` that uses the handler above.
21–25	21 <type name="Codilar\CategoryCache\Model\CategoryDetails"> 22 <arguments> 23 <argument name="logger" xsi:type="object">Codilar\CategoryCache\Logger\Logger</argument> 24 </arguments> 25 </type>	Gives Model\CategoryDetails our logger in place of the default system logger, so HIT/MISS lines go to their own file.
26–27	26 27 <!-- CLI commands (service proxied so it is only built when a command runs) -->	Blank line and a comment.
28–37	28 <type name="Codilar\CategoryCache\Console\Command\GetCommand"> 29 <arguments> 30 <argument name="categoryDetails" xsi:type="object">Codilar\CategoryCache\Api\CategoryDetailsInterface\Proxy</argument> 31 </arguments> 32 </type> 33 <type name="Codilar\CategoryCache\Console\Command\CleanCommand"> 34 <arguments> 35 <argument name="categoryDetails" xsi:type="object">Codilar\CategoryCache\Api\CategoryDetailsInterface\Proxy</argument> 36 </arguments> 37 </type>	Both commands receive the service's **Proxy** (a class Magento generates under generated/). `bin/magento` builds every command on startup. The proxy postpones building the real service, with its repository and cache dependencies, until a command actually calls it.
38–45	38 <type name="Magento\Framework\Console\CommandListInterface"> 39 <arguments> 40 <argument name="commands" xsi:type="array"> 41 <item name="codilar_category_get" xsi:type="object">Codilar\CategoryCache\Console\Command\GetCommand</item> 42 <item name="codilar_category_cache_clean" xsi:type="object">Codilar\CategoryCache\Console\Command\CleanCommand</item> 43 </argument> 44 </arguments> 45 </type>	Adds both commands to Magento's CLI command list. This is how they show up in `bin/magento list`.
46	46 </config>	Closing tag.

10 Console/Command/GetCommand.php

> `bin/magento codilar:category:get <id>`: the demo command. It calls the service several times and prints HIT or MISS with the time each call took, which shows that cached data is reused.

Lines	Code	What it does
1–8	<?php /** * bin/magento codilar:category:get <id> [--store=<id>] [--times=<n>] * * Fetches the category details <n> times (default 2) and reports whether each * call was served from the cache or from the database, with timings. */ declare(strict_types=1);	PHP open tag, a docblock showing usage, strict types.
9–10	 namespace Codilar\CategoryCache\Console\Command;	Namespace.
11–23	 use Codilar\CategoryCache\Api\CategoryDetailsInterface; use Magento\Framework\App\Area; use Magento\Framework\App\State; use Magento\Framework\Console\Cli; use Magento\Store\Model\App\Emulation; use Magento\Store\Model\StoreManagerInterface; use Symfony\Component\Console\Command\Command; use Symfony\Component\Console\Input\InputArgument; use Symfony\Component\Console\Input\InputInterface; use Symfony\Component\Console\Input\InputOption; use Symfony\Component\Console\Output\OutputInterface; 	Imports: our service; Area and State for setting the area code; Cli for return codes; Emulation for store emulation; StoreManager for resolving the store; and Symfony Console classes for arguments, options and output.
24–25	class GetCommand extends Command {	Magento CLI commands are Symfony Console commands.
26–28	private const ARG_ID = 'category_id'; private const OPT_STORE = 'store'; private const OPT_TIMES = 'times';	Names of the argument and options, defined once to avoid typos.
29–36	 /** * @param CategoryDetailsInterface $categoryDetails Injected as a Proxy (see di.xml) * @param StoreManagerInterface $storeManager * @param Emulation $emulation * @param State $appState * @param string|null $name */	Constructor docblock.

37–45	public function __construct(
	private readonly CategoryDetailsInterface $categoryDetails,
	private readonly StoreManagerInterface $storeManager,
	private readonly Emulation $emulation,
	private readonly State $appState,
	?string $name = null
	) {
	parent::__construct($name);
	}
		Constructor: DI supplies the service (as a Proxy, see di.xml), the store manager, the emulation service and the app state. `parent::__construct()` is required by Symfony.

46–51	 /** * @inheritDoc */ protected function configure(): void {	configure() defines the command's name and inputs.
52–53	$this->setName('codilar:category:get') ->setDescription('Get category ID, name and URL (cached for 1 hour) and show cache hits/misses.')	Command name and the description shown in `bin/magento list`.
54	->addArgument(self::ARG_ID, InputArgument::REQUIRED, 'Category ID')	Required argument: the category ID.
55	->addOption(self::OPT_STORE, 's', InputOption::VALUE_REQUIRED, 'Store view ID (default store view if omitted)')	Optional `--store` / `-s`: which store view to use.
56	->addOption(self::OPT_TIMES, 't', InputOption::VALUE_REQUIRED, 'How many times to fetch', '2');	Optional `--times` / `-t`: how many calls to make (default 2: one MISS followed by a HIT when the cache is cold).

Lines	Code	What it does

57–59	`        parent::configure();`
	`    }`
	
		Calls the parent configure() and closes the method.

60–64	`    /**`
	`     * @inheritDoc`
	`     */`
	`    protected function execute(InputInterface $input, OutputInterface $output): int`
	`    {`
		execute() runs when the command is invoked.

65	`        $categoryId = (int)$input->getArgument(self::ARG_ID);`	Reads the category ID and casts it to int.

66–69	`        $storeOption = $input->getOption(self::OPT_STORE);`
	`        $storeId = $storeOption !== null`
	`            ? (int)$this->storeManager->getStore($storeOption)->getId()`
	`            : (int)$this->storeManager->getDefaultStoreView()->getId();`
		Resolves the store view ID: the one passed in `--store`, otherwise the default store view. CLI has no current storefront store, so we pick one explicitly.

70–71	`        $times = max(1, (int)$input->getOption(self::OPT_TIMES));` 	Number of calls, at least 1.

72–76	`        try {`
	`            return $this->appState->emulateAreaCode(`
	`                Area::AREA_FRONTEND,`
	`                fn (): int => $this->fetch($output, $categoryId, $storeId, $times)`
	`            );`
		CLI runs with no area code. Building a category URL needs one, so `emulateAreaCode(frontend, ...)` runs fetch() as if inside a storefront request. Without it Magento throws "Area code is not set" (we hit this while testing).

77–82	`        } catch (\Throwable $e) {`
	`            $output->writeln('<error>' . $e->getMessage() . '</error>');`
	`            return Cli::RETURN_FAILURE;`
	`        }`
	`    }`
		Any error, such as a category ID that doesn't exist, is printed in red and the command exits with code 1.

83–92	`    /**`
	`     * Fetch the details $times times inside a storefront emulation.`
	`     *`
	`     * @param OutputInterface $output`
	`     * @param int $categoryId`
	`     * @param int $storeId`
	`     * @param int $times`
	`     * @return int`
	`     */`
		Docblock for the private fetch() helper.

93–94	`    private function fetch(OutputInterface $output, int $categoryId, int $storeId, int $times): int` `    {`	fetch() signature.
95–96	`        // Store emulation so category URLs are built for the storefront, not the CLI/admin context.` `        $this->emulation->startEnvironmentEmulation($storeId, Area::AREA_FRONTEND, true);`	**Store emulation**: switches config, design and URL settings to the selected store view, so getUrl() returns the storefront URL (e.g. `http://localhost:8083/test-category.html`) and not an admin or CLI URL.
97–98	`        try {` `            for ($i = 1; $i <= $times; $i++) {`	Loops `$times` times.

99–101	`                $start = microtime(true);`
	`                $details = $this->categoryDetails->get($categoryId, $storeId);`
	`                $ms = (microtime(true) - $start) * 1000;`
		Times each `get()` call in milliseconds. A MISS takes tens of ms because it queries the database. A HIT takes well under 1 ms.

102–110	
	`                $output->writeln(sprintf(`
	`                    'Call #%d: %s (%.2f ms)',`
	`                    $i,`
	`                    $this->categoryDetails->wasLastHit()`
	`                        ? '<info>CACHE HIT</info>'`
	`                        : '<comment>CACHE MISS - loaded from database</comment>',`
	`                    $ms`
	`                ));`
		Prints `Call #n: CACHE HIT` (green) or `CACHE MISS - loaded from database` (yellow), using wasLastHit(), along with the duration.

Lines	Code	What it does
111–117	111 $output->writeln(sprintf(112 ' ID: %d | Name: %s | URL: %s', 113 $details['id'], 114 $details['name'], 115 $details['url'] 116)); 117 }	Prints the fetched ID, name and URL.
118–120	118 } finally { 119 $this->emulation->stopEnvironmentEmulation(); 120 }	`finally` always ends the store emulation, even after an error, so the environment is restored.
121–124	121 122 return Cli::RETURN_SUCCESS; 123 } 124 }	Returns success (exit code 0) and closes the class.

11 Console/Command/CleanCommand.php

> `bin/magento codilar:category:cache:clean <id>` clears the cache for one category from the command line.

Lines	Code	What it does
1–8	1 <?php 2 /** 3 * bin/magento codilar:category:cache:clean <id> - drop cached details of one category. 4 */ 5 declare(strict_types=1); 6 7 namespace Codilar\CategoryCache\Console\Command; 8	PHP open tag, docblock, strict types, namespace.
9–15	9 use Codilar\CategoryCache\Api\CategoryDetailsInterface; 10 use Magento\Framework\Console\Cli; 11 use Symfony\Component\Console\Command\Command; 12 use Symfony\Component\Console\Input\InputArgument; 13 use Symfony\Component\Console\Input\InputInterface; 14 use Symfony\Component\Console\Output\OutputInterface; 15	Imports: our service, Cli return codes, Symfony Console classes.
16–18	16 class CleanCommand extends Command 17 { 18 private const ARG_ID = 'category_id';	Class declaration and the argument-name constant.
19–30	19 20 /** 21 * @param CategoryDetailsInterface $categoryDetails Injected as a Proxy (see di.xml) 22 * @param string|null $name 23 */ 24 public function __construct(25 private readonly CategoryDetailsInterface $categoryDetails, 26 ?string $name = null 27) { 28 parent::__construct($name); 29 } 30	Constructor: DI supplies the service (as a Proxy).
31–41	31 /** 32 * @inheritDoc 33 */ 34 protected function configure(): void 35 { 36 $this->setName('codilar:category:cache:clean') 37 ->setDescription('Clear the cached details of one category (all store views).') 38 ->addArgument(self::ARG_ID, InputArgument::REQUIRED, 'Category ID'); 39 parent::configure(); 40 } 41	configure(): command name, description and the required category_id argument.

Lines	Code	What it does
42–46	42 /** 43 * @inheritDoc 44 */ 45 protected function execute(InputInterface $input, OutputInterface $output): int 46 {	execute() signature.
47–48	47 $categoryId = (int)$input->getArgument(self::ARG_ID); 48 $this->categoryDetails->clean($categoryId);	Reads the ID and calls `clean()`, which removes the entries tagged `CODILAR_CATEGORY_DETAILS_<id>` in every store view.
49–53	49 $output->writeln(sprintf('<info>Cleared cached details for category %d.</info>', $categoryId)); 50 51 return Cli::RETURN_SUCCESS; 52 } 53 }	Prints a confirmation and returns success.